

1. HTML Interview Questions

Q1. What is HTML?

Answer:

HTML stands for **HyperText Markup Language**. It is used to create the structure of a web page.

HTML tells the browser what content to display, such as:

- Headings
- Paragraphs
- Images
- Links
- Tables
- Forms

Example:

Q2. What is the difference between HTML and HTML5?

Answer:

HTML5 is the latest version of HTML. It introduced new semantic tags and features.

Important HTML5 features include:

- Semantic tags like <header>, <footer>, <section>, <article>
- Audio and video support
- Canvas for graphics
- Local storage
- Better form input types like email, date, number

Example:

Q3. What are semantic HTML tags?

Answer:

Semantic tags clearly describe their meaning to the browser and developer.

Examples:

They improve:

- Code readability
- SEO
- Accessibility

Example:

Q4. Difference between <div> and ?

Answer:

<div> is a **block-level element**, while is an **inline element**.

div starts on a new line and takes full width.

span stays in the same line and takes only required width.

Q5. What is the difference between id and class?

Answer:

id is unique and should be used for one element only.

class can be used for multiple elements.

In CSS:

```
1  #header {  
2    color: blue;  
3  }  
4  
5  .box {  
6    color: red;  
7  }
```

Q6. What are HTML forms?

Answer:

HTML forms are used to collect user input.

Example:

Common form elements:

- <input>
 - <textarea>
 - <select>
 - <button>
 - <label>
-

Q7. What is the difference between GET and POST methods?

Answer:

GET sends data in the URL.

It is used for search or filter operations.

POST sends data in the request body.

It is used for submitting sensitive or large data like login forms.

Main differences:

- GET data is visible in URL
 - POST data is not visible in URL
 - GET is less secure than POST
 - POST is better for form submission
-

Q8. What is the use of the alt attribute in images?

Answer:

The alt attribute provides alternative text for an image.

Example:

It is useful for:

- Accessibility
 - SEO
 - Showing text if image fails to load
-

2. CSS Interview Questions

Q9. What is CSS?

Answer:

CSS stands for **Cascading Style Sheets**. It is used to style HTML elements.

CSS controls:

- Colors
- Fonts
- Layout
- Spacing
- Responsiveness
- Animations

Example:

```
1  h1 {  
2    color: blue;  
3    font-size: 30px;  
4  }
```

Q10. What are the types of CSS?

Answer:

There are three types of CSS:

1. Inline CSS

Written inside the HTML element.

2. Internal CSS

Written inside the <style> tag.

3. External CSS

Written in a separate .css file.

External CSS is preferred because it keeps code clean and reusable.

Q11. What is the CSS box model?

Answer:

The CSS box model describes how every HTML element is displayed as a rectangular box.

It contains:

1. Content
2. Padding
3. Border
4. Margin

Example:

```
1 .box {  
2   width: 200px;  
3   padding: 20px;  
4   border: 2px solid black;  
5   margin: 10px;  
6 }
```

Explanation:

- **Content:** actual text or image
 - **Padding:** space inside the border
 - **Border:** line around padding and content
 - **Margin:** space outside the border
-

Q12. Difference between margin and padding?

Answer:

margin is the space outside an element.

padding is the space inside an element, between content and border.

Example:

```
1 .box {  
2   margin: 20px;  
3   padding: 10px;  
4 }
```

Q13. Difference between relative, absolute, fixed, and sticky position?

Answer:

relative

Element moves relative to its normal position.

```
1 .box {
2   position: relative;
3   top: 10px;
4 }
```

absolute

Element is positioned relative to its nearest positioned parent.

```
1 .box {
2   position: absolute;
3   top: 0;
4   left: 0;
5 }
```

fixed

Element stays fixed relative to the browser window.

```
1 .header {
2   position: fixed;
3   top: 0;
4 }
```

sticky

Element behaves like relative until a scroll position, then becomes fixed.

```
1 .nav {
2   position: sticky;
3   top: 0;
4 }
```

Q14. What is Flexbox?

Answer:

Flexbox is a CSS layout system used to arrange elements in one direction: row or column.

Example:

```
1 .container {
2   display: flex;
3   justify-content: center;
4   align-items: center;
5 }
```

Important properties:

- display: flex
 - justify-content
 - align-items
 - flex-direction
 - gap
 - flex-wrap
-

Q15. Difference between Flexbox and Grid?

Answer:

Flexbox is mainly used for **one-dimensional layouts**.

That means either row or column.

CSS Grid is used for **two-dimensional layouts**.

That means rows and columns together.

Example Flexbox:

```
1 .container {  
2   display: flex;  
3 }
```

Example Grid:

```
1 .container {  
2   display: grid;  
3   grid-template-columns: repeat(3, 1fr);  
4 }
```

Use Flexbox for navbar, buttons, cards in a row.

Use Grid for full page layouts and gallery layouts.

Q16. What is media query?

Answer:

Media query is used to make websites responsive for different screen sizes.

Example:

```
1 @media (max-width: 768px) {  
2   body {  
3     background-color: lightgray;
```

```
4    }
5
6    .container {
7        flex-direction: column;
8    }
9 }
```

This CSS applies when screen width is 768px or less.

Q17. What is CSS specificity?

Answer:

Specificity decides which CSS rule will be applied when multiple rules target the same element.

Priority order:

1. Inline style
2. ID selector
3. Class selector
4. Element selector

Example:

```
1  p {
2      color: blue;
3  }
4
5  .text {
6      color: green;
7  }
8
9  #para {
10     color: red;
11 }
```

The color will be **red** because ID has higher specificity.

3. JavaScript Interview Questions

Q18. What is JavaScript?

Answer:

JavaScript is a scripting language used to make web pages interactive.

It is used for:

- Form validation
- Dynamic content
- API calls
- DOM manipulation
- Events
- Animations

Example:

```
1 console.log("Hello JavaScript");
```

Q19. Difference between var, let, and const?

Answer:

var is function-scoped and can be redeclared.

```
1 var name = "Vijay";  
2 var name = "Rahul";
```

let is block-scoped and can be updated but not redeclared in the same scope.

```
1 let age = 25;  
2 age = 26;
```

const is block-scoped and cannot be reassigned.

```
1 const city = "Pune";
```

In modern JavaScript, prefer let and const.

Q20. What are data types in JavaScript?

Answer:

JavaScript has primitive and non-primitive data types.

Primitive data types:

```
1 let name = "Vijay";    // string
2 let age = 25;           // number
3 let isActive = true;   // boolean
4 let value = null;      // null
5 let x;                 // undefined
6 let id = Symbol();     // symbol
7 let bigNumber = 123n;  // bigint
```

Non-primitive data type:

```
1 let user = {
2   name: "Vijay",
3   age: 25
4 };
5
6 let numbers = [1, 2, 3];
```

Objects and arrays are non-primitive.

Q21. Difference between == and ===?

Answer:

== compares values after type conversion.

```
1 5 == "5"; // true
```

=== compares both value and data type.

```
1 5 === "5"; // false
```

In interviews, say:

Always prefer === because it avoids unexpected type conversion.

Q22. What is a function in JavaScript?

Answer:

A function is a reusable block of code that performs a specific task.

Example:

```
1 function add(a, b) {
2   return a + b;
3 }
4
5 console.log(add(5, 3));
```

Functions help in:

- Code reuse
 - Better organization
 - Avoiding repetition
-

Q23. What is an arrow function?

Answer:

Arrow function is a shorter syntax for writing functions.

Normal function:

```
1 function add(a, b) {  
2   return a + b;  
3 }
```

Arrow function:

```
1 const add = (a, b) => {  
2   return a + b;  
3 };
```

Short version:

```
1 const add = (a, b) => a + b;
```

Q24. What is DOM?

Answer:

DOM stands for **Document Object Model**.

It represents the HTML page as a tree structure. JavaScript can use DOM to change HTML and CSS dynamically.

Example:

```
1 document.getElementById("title").innerText = "New Title";
```

HTML:

Q25. What are JavaScript events?

Answer:

Events are actions that happen in the browser.

Examples:

- Click
- Mouseover
- Keypress
- Submit
- Load

Example:

```
1 document.getElementById("btn").addEventListener("click",  
function() {  
2     alert("Button clicked");  
3 });
```

Q26. Difference between null and undefined?

Answer:

undefined means a variable is declared but value is not assigned.

```
1 let x;  
2 console.log(x); // undefined
```

null means intentional empty value.

```
1 let y = null;  
2 console.log(y); // null
```

Q27. What is an array in JavaScript?

Answer:

An array is used to store multiple values in a single variable.

Example:

```
1 let fruits = ["Apple", "Banana", "Mango"];  
2  
3 console.log(fruits[0]); // Apple
```

Common array methods:

```
1 fruits.push("Orange");  
2 fruits.pop();  
3 fruits.length;
```

Q28. What is an object in JavaScript?

Answer:

An object stores data in key-value pairs.

Example:

```
1  let user = {  
2    name: "Vijay",  
3    age: 25,  
4    city: "Pune"  
5  };  
6  
7  console.log(user.name);
```

Objects are used to represent real-world entities.

Q29. What is callback function?

Answer:

A callback function is a function passed as an argument to another function.

Example:

```
1  function greet(name, callback) {  
2    console.log("Hello " + name);  
3    callback();  
4  }  
5  
6  function sayBye() {  
7    console.log("Bye");  
8  }  
9  
10 greet("Vijay", sayBye);
```

Callbacks are commonly used in:

- Events
 - API calls
 - Timers
 - Asynchronous programming
-

Q30. What is the difference between synchronous and asynchronous JavaScript?

Answer:

Synchronous code runs line by line.

```
1 console.log("First");
2 console.log("Second");
3 console.log("Third");
```

Asynchronous code allows some tasks to run later without blocking other code.

```
1 console.log("First");
2
3 setTimeout(() => {
4   console.log("Second");
5 }, 1000);
6
7 console.log("Third");
```

Output:

```
1 First
2 Third
3 Second
```

Q31. What is Promise in JavaScript?

Answer:

A Promise is used to handle asynchronous operations.

A Promise has three states:

1. Pending
2. Fulfilled
3. Rejected

Example:

```
1 let promise = new Promise((resolve, reject) => {
2   let success = true;
3
4   if (success) {
5     resolve("Task completed");
6   } else {
```

```
7     reject("Task failed");
8   }
9 });
10
11 promise
12   .then(result => console.log(result))
13   .catch(error => console.log(error));
```

Q32. What is async/await?

Answer:

async and await are used to handle Promises in a cleaner way.

Example:

```
1  async function fetchData() {
2    try {
3      let response = await
fetch("https://api.example.com/users");
4      let data = await response.json();
5      console.log(data);
6    } catch (error) {
7      console.log(error);
8    }
9  }
```

await pauses the function until the Promise is resolved.

Q33. What is hoisting?

Answer:

Hoisting means JavaScript moves declarations to the top of their scope before execution.

Example with var:

```
1  console.log(a);
2  var a = 10;
```

Output:

```
1  undefined
```

Because declaration is hoisted, but value assignment is not.

With let and const, accessing before declaration gives an error.

```
1 console.log(b);
2 let b = 20;
```

This gives ReferenceError.

Q34. What is closure?

Answer:

A closure is created when an inner function remembers variables from its outer function, even after the outer function has finished execution.

Example:

```
1 function outer() {
2   let count = 0;
3
4   return function inner() {
5     count++;
6     console.log(count);
7   };
8 }
9
10 let counter = outer();
11
12 counter(); // 1
13 counter(); // 2
14 counter(); // 3
```

Here, inner() remembers the count variable.

Q35. What is event bubbling?

Answer:

Event bubbling means when an event happens on a child element, it first runs on the child, then moves upward to its parent elements.

Example:

```
1 document.getElementById("parent").addEventListener("click",
2   function() {
3     console.log("Parent clicked");
4   });
```

```
4
5 document.getElementById("child").addEventListener("click",
function() {
6     console.log("Child clicked");
7 });
```

If button is clicked, output:

```
1 Child clicked
2 Parent clicked
```

4. Common Practical Interview Questions

Q36. How do you center a div?

Answer using Flexbox:

```
1 .parent {
2     display: flex;
3     justify-content: center;
4     align-items: center;
5     height: 100vh;
6 }
```

Q37. How do you hide an element in CSS?

Answer:

There are multiple ways.

```
1 display: none;
```

This removes the element from layout.

```
1 visibility: hidden;
```

This hides the element but keeps its space.

```
1 opacity: 0;
```

This makes the element transparent but it still exists and takes space.

Q38. How do you add JavaScript to HTML?

Answer:

JavaScript can be added in three ways.

1. Inline
2. Internal
3. External

External JavaScript is preferred for clean and reusable code.

Q39. How do you select elements in JavaScript?

Answer:

Common DOM selectors are:

```
1 document.getElementById("title");
2 document.getElementsByClassName("box");
3 document.getElementsByTagName("p");
4 document.querySelector(".box");
5 document.querySelectorAll(".box");
```

Most commonly used:

```
1 document.querySelector("#title");
2 document.querySelector(".box");
```

Q40. How do you validate a form using JavaScript?

Answer:

Example:

```
1 function validateForm() {
2     let name = document.getElementById("name").value;
3
4     if (name === "") {
5         alert("Name is required");
6         return false;
7     }
8
9     return true;
10 }
```

Q1. What is `<!DOCTYPE html>`?

Answer:

`<!DOCTYPE html>` tells the browser that the document is written in **HTML5**.

It is written at the very top of the HTML file.

It is not an HTML tag. It is a declaration.

If we do not write it, the browser may go into **quirks mode**, where some CSS and layout may behave differently.

Interview line:

`<!DOCTYPE html>` tells the browser to render the page using HTML5 standards mode.

Q2. What is the basic structure of an HTML document?

Answer:

A basic HTML document has:

Explanation:

- `<!DOCTYPE html>` defines HTML5 document type
 - `<html>` is the root element
 - `<head>` contains metadata, title, CSS links
 - `<body>` contains visible page content
-

Q3. What is the use of the `<head>` tag?

Answer:

The `<head>` tag contains information about the web page, but it is not directly visible on the page.

It contains:

Common things inside `<head>`:

- Page title
- Meta tags
- CSS file links
- Favicon
- SEO information

- Scripts sometimes
-

Q4. What is the use of `<meta charset="UTF-8">`?

Answer:

`<meta charset="UTF-8">` tells the browser which character encoding to use.

UTF-8 supports almost all characters, including:

- English letters
- Numbers
- Symbols
- Emojis
- Indian languages
- Special characters

Example:

Interview line:

It helps the browser display text correctly.

Q5. What is the viewport meta tag?

Answer:

The viewport meta tag helps make a website responsive on mobile devices.

Explanation:

- `width=device-width` sets the page width equal to device screen width
- `initial-scale=1.0` sets the default zoom level

Without this tag, mobile browsers may show the desktop version zoomed out.

Q6. What is the difference between `<head>` and `<body>`?

Answer:

`<head>` contains page information and settings.

Example:

`<body>` contains visible content.

Example:

Simple answer:

Head contains metadata. Body contains visible page content.

Q7. What is the difference between `<script>`, *async*, and *defer*?

Answer:

Normal script:

It blocks HTML parsing until the script is downloaded and executed.

With *defer*:

The script downloads in the background and runs after HTML parsing is complete.

With *async*:

The script downloads in the background and runs as soon as it is ready.

Interview answer:

- Use *defer* when script depends on HTML elements.
 - Use *async* for independent scripts like analytics.
-

Q8. What is the difference between `<strong>` and `<b>`?

Answer:

Both make text bold, but their meaning is different.

`<b>` is used only for visual bold styling.

`<strong>` means the text is important semantically.

Interview line:

strong gives importance to text, while *b* only makes text bold.

Q9. What is the difference between `<em>` and `<i>`?

Answer:

Both usually display italic text.

`<i>` is used only for styling or alternate text.

`<em>` gives emphasis to the text semantically.

Q10. What are HTML entities?

Answer:

HTML entities are used to display reserved characters.

Example:

Common entities:

- `<` = <
 - `>` = >
 - `&` = &
 - `©` = ©
 - ` ` = non-breaking space
-

Q11. What is the use of the *label* tag?

Answer:

The `<label>` tag describes a form input.

When the user clicks the label, the input gets focused.

It improves:

- Accessibility
 - User experience
 - Form readability
-

Q12. What is the difference between *required*, *readonly*, and *disabled*?

Answer:

required means the input must be filled.

readonly means the user cannot edit the value, but the value is submitted.

disabled means the user cannot edit it, and the value is not submitted.

Q13. What is the use of *data-** attributes?

Answer:

*data-** attributes are used to store custom data in HTML elements.

JavaScript:

```
1 const button = document.querySelector("button");
2 console.log(button.dataset.id);
```

Output:

```
1 101
```

Interview line:

*data-** attributes store custom information that can be accessed using JavaScript.

Q14. What is accessibility in HTML?

Answer:

Accessibility means making a website usable for everyone, including people with disabilities.

Examples:

Good accessibility includes:

- Proper *alt* text
 - Labels for inputs
 - Semantic HTML
 - Keyboard navigation
 - Good color contrast
-

Q15. What is the use of *target="_blank"*?

Answer:

target="_blank" opens a link in a new tab.

For security, we should also use:

rel="noopener noreferrer" helps prevent security issues when opening a new tab.

More Important CSS Interview Questions

Q16. What is *box-sizing: border-box*?

Answer:

By default, width only applies to content. Padding and border are added extra.

```
1 .box {  
2   width: 200px;  
3   padding: 20px;  
4   border: 5px solid black;  
5 }
```

Actual width becomes more than 200px.

With *border-box*:

```
1 .box {  
2   box-sizing: border-box;  
3   width: 200px;  
4   padding: 20px;  
5   border: 5px solid black;  
6 }
```

Now total width remains 200px.

Interview line:

box-sizing: border-box includes content, padding, and border inside the given width and height.

Q17. What is CSS reset or normalize CSS?

Answer:

Different browsers apply different default styles.

CSS reset removes default browser styling.

Example:

```
1 * {  
2   margin: 0;  
3   padding: 0;  
4   box-sizing: border-box;  
5 }
```

Normalize CSS makes default styles consistent across browsers.

Simple answer:

CSS reset or normalize is used to make styling consistent across different browsers.

Q18. What are pseudo-classes in CSS?

Answer:

Pseudo-classes define a special state of an element.

Examples:

```
1  a:hover {
2    color: red;
3  }
4
5  input:focus {
6    border-color: blue;
7  }
8
9  li:first-child {
10   font-weight: bold;
11 }
```

Common pseudo-classes:

- *:hover*
 - *:focus*
 - *:active*
 - *:first-child*
 - *:last-child*
 - *:nth-child()*
-

Q19. What are pseudo-elements in CSS?

Answer:

Pseudo-elements style a specific part of an element.

Examples:

```
1  p::first-letter {
2    font-size: 30px;
3  }
```

```
4
5 .box::before {
6     content: "★";
7 }
8
9 .box::after {
10    content: "√";
11 }
```

Common pseudo-elements:

- *::before*
 - *::after*
 - *::first-letter*
 - *::first-line*
-

Q20. Difference between pseudo-class and pseudo-element?

Answer:

Pseudo-class targets a state.

```
1 button:hover {
2     background-color: blue;
3 }
```

Pseudo-element targets a part of an element.

```
1 p::first-letter {
2     color: red;
3 }
```

Interview line:

Pseudo-class selects element state, pseudo-element selects part of an element.

Q21. What is z-index?

Answer:

z-index controls the stacking order of positioned elements.

```
1 .box1 {
2     position: absolute;
3     z-index: 1;
4 }
```

```
5
6 .box2 {
7     position: absolute;
8     z-index: 2;
9 }
```

The element with higher *z-index* appears on top.

Important: *z-index* works mainly with positioned elements like:

```
1 position: relative;
2 position: absolute;
3 position: fixed;
4 position: sticky;
```

Q22. Difference between *em*, *rem*, *%*, *px*, and *vh/vw*?

Answer:

px is fixed size.

```
1 font-size: 16px;
```

em depends on parent element font size.

```
1 font-size: 2em;
```

rem depends on root HTML font size.

```
1 font-size: 2rem;
```

% depends on parent size.

```
1 width: 50%;
```

vh and *vw* depend on viewport size.

```
1 height: 100vh;
```

```
2 width: 100vw;
```

Interview tip:

For responsive design, *rem*, *%*, *vh*, and *vw* are commonly used.

Q23. What are CSS variables?

Answer:

CSS variables store reusable values.

```
1 :root {
2     --primary-color: blue;
```

```
3   --main-padding: 20px;
4 }
5
6 button {
7   background-color: var(--primary-color);
8   padding: var(--main-padding);
9 }
```

Benefits:

- Reusability
 - Easy theme changes
 - Cleaner code
-

Q24. Difference between transition and animation?

Answer:

transition changes style smoothly when a state changes.

```
1 button {
2   background-color: blue;
3   transition: background-color 0.3s ease;
4 }
5
6 button:hover {
7   background-color: red;
8 }
```

animation can run automatically using keyframes.

```
1 @keyframes moveBox {
2   from {
3     left: 0;
4   }
5   to {
6     left: 100px;
7   }
8 }
9
10 .box {
11   position: relative;
12   animation: moveBox 2s infinite;
13 }
```

Simple answer:

Transition needs a trigger like hover. Animation can run automatically and has multiple steps.

More Important JavaScript Interview Questions

Q25. What is *"use strict"* in JavaScript?

Answer:

"use strict" enables strict mode in JavaScript.

```
1  "use strict";  
2  
3  x = 10;
```

This will give an error because *x* is not declared.

Strict mode helps catch common coding mistakes.

Interview line:

Strict mode makes JavaScript code more secure and less error-prone.

Q26. What are truthy and falsy values?

Answer:

Falsy values are values treated as false in conditions.

Falsy values:

```
1  false  
2  0  
3  ""  
4  null  
5  undefined  
6  NaN
```

Example:

```
1  if ("hello") {  
2    console.log("Truthy");  
3  }  
4
```

```
5  if ("" ) {  
6    console.log("This will not run");  
7  }
```

Everything else is generally truthy.

Q27. What is template literal?

Answer:

Template literals are used to write strings easily using backticks.

```
1  let name = "Vijay";  
2  let age = 25;  
3  
4  let message = `My name is ${name} and I am ${age} years old.`;  
5  
6  console.log(message);
```

Benefits:

- Easy variable insertion
 - Multi-line strings
 - Cleaner syntax
-

Q28. What is destructuring in JavaScript?

Answer:

Destructuring is used to extract values from arrays or objects.

Array destructuring:

```
1  let numbers = [10, 20, 30];  
2  
3  let [a, b] = numbers;  
4  
5  console.log(a); // 10  
6  console.log(b); // 20
```

Object destructuring:

```
1  let user = {  
2    name: "Vijay",  
3    city: "Pune"  
4  };
```

```
5
6 let { name, city } = user;
7
8 console.log(name);
9 console.log(city);
```

Q29. What is spread operator?

Answer:

The spread operator expands array or object values.

Array example:

```
1 let arr1 = [1, 2];
2 let arr2 = [3, 4];
3
4 let result = [...arr1, ...arr2];
5
6 console.log(result);
```

Output:

```
1 [1, 2, 3, 4]
```

Object example:

```
1 let user = {
2   name: "Vijay"
3 };
4
5 let updatedUser = {
6   ...user,
7   city: "Pune"
8 };
```

Q30. What is rest parameter?

Answer:

Rest parameter collects multiple values into an array.

```
1 function add(...numbers) {
2   return numbers.reduce((total, num) => total + num, 0);
3 }
4
```

```
5 console.log(add(10, 20, 30));
```

Output:

```
1 60
```

Difference:

- Spread expands values
 - Rest collects values
-

Q31. Difference between *map*, *filter*, and *reduce*?

Answer:

map() transforms each item and returns a new array.

```
1 let numbers = [1, 2, 3];
2
3 let doubled = numbers.map(num => num * 2);
4
5 console.log(doubled);
```

Output:

```
1 [2, 4, 6]
```

filter() returns items that match a condition.

```
1 let numbers = [1, 2, 3, 4];
2
3 let even = numbers.filter(num => num % 2 === 0);
4
5 console.log(even);
```

Output:

```
1 [2, 4]
```

reduce() returns a single value.

```
1 let numbers = [1, 2, 3];
2
3 let total = numbers.reduce((sum, num) => sum + num, 0);
4
5 console.log(total);
```

Output:

```
1 6
```

Q32. What is *this* keyword in JavaScript?

Answer:

this refers to the object that is calling the function.

Example:

```
1 let user = {  
2   name: "Vijay",  
3   greet: function() {  
4     console.log(this.name);  
5   }  
6 };  
7  
8 user.greet();
```

Output:

```
1 Vijay
```

Here, *this* refers to the *user* object.

In arrow functions, *this* behaves differently. Arrow functions do not have their own *this*; they use *this* from the surrounding scope.

Q33. What is event delegation?

Answer:

Event delegation means adding an event listener to a parent element instead of adding it to each child.

Example:

```
1 document.getElementById("list").addEventListener("click",  
function(event) {  
2   if (event.target.tagName === "LI") {  
3     console.log(event.target.innerText);  
4   }  
5 });
```

Benefits:

- Better performance
 - Useful for dynamically added elements
 - Less code
-

Q34. What is *preventDefault()*?

Answer:

preventDefault() stops the default behavior of an element.

Example:

```
1 document.querySelector("form").addEventListener("submit",
function(event) {
2   event.preventDefault();
3   console.log("Form submission stopped");
4 });
```

For example, it can stop:

- Form submission
 - Link navigation
 - Checkbox default behavior
-

Q35. What is *stopPropagation()*?

Answer:

stopPropagation() stops an event from bubbling up to parent elements.

```
1 document.getElementById("child").addEventListener("click",
function(event) {
2   event.stopPropagation();
3   console.log("Child clicked");
4 });
```

If used, parent click event will not run.

Q36. Difference between *localStorage*, *sessionStorage*, and *cookies*?

Answer:

localStorage stores data permanently until manually removed.

```
1 localStorage.setItem("name", "Vijay");
2 localStorage.getItem("name");
```

sessionStorage stores data only for one browser tab session.

```
1 sessionStorage.setItem("token", "abc123");
```

Cookies store small data and can be sent with server requests.

```
1 document.cookie = "user=Vijay";
```

Simple answer:

- *localStorage*: persists even after browser close
 - *sessionStorage*: clears when tab is closed
 - *cookies*: small data, often used for authentication/tracking
-

Q37. What is JSON?

Answer:

JSON stands for **JavaScript Object Notation**.

It is a lightweight format used to store and exchange data.

Example JSON:

```
1 {  
2   "name": "Vijay",  
3   "city": "Pune"  
4 }
```

Convert object to JSON string:

```
1 let user = {  
2   name: "Vijay",  
3   city: "Pune"  
4 };  
5  
6 let jsonData = JSON.stringify(user);
```

Convert JSON string to object:

```
1 let obj = JSON.parse(jsonData);
```

Q38. What is error handling in JavaScript?

Answer:

Error handling is done using *try...catch*.

```
1 try {  
2   let result = JSON.parse("invalid json");  
3   console.log(result);  
4 } catch (error) {
```

```
5 console.log("Something went wrong");
6 }
```

It prevents the whole program from crashing when an error occurs.

Q39. What is debouncing?

Answer:

Debouncing means delaying function execution until the user stops performing an action.

Example use case:

- Search box
- Resize event
- Input validation

Example:

```
1 let timer;
2
3 function searchData() {
4   clearTimeout(timer);
5
6   timer = setTimeout(() => {
7     console.log("API call after user stops typing");
8   }, 500);
9 }
```

If the user keeps typing, the function waits.

Q40. What is throttling?

Answer:

Throttling means running a function at fixed intervals, even if the event happens many times.

Example use case:

- Scroll event
- Window resize
- Mouse movement

Simple explanation:

Debouncing waits until the user stops. Throttling runs after a fixed time interval.

Very Common Quick Questions

Q41. What is the difference between *innerHTML*, *innerText*, and *textContent*?

Answer:

innerHTML gets or sets HTML content.

```
1 element.innerHTML = "<b>Hello</b>";
```

innerText gets visible text only.

```
1 element.innerText = "Hello";
```

textContent gets all text content, including hidden text.

```
1 element.textContent = "Hello";
```

Interview tip:

Use *textContent* when inserting plain text. Use *innerHTML* carefully because it can create security risks if used with user input.

Q42. Difference between *querySelector* and *querySelectorAll*?

Answer:

querySelector() returns the first matching element.

```
1 document.querySelector(".box");
```

querySelectorAll() returns all matching elements.

```
1 document.querySelectorAll(".box");
```

Q43. What is NaN?

Answer:

NaN means **Not a Number**.

Example:

```
1 let result = "hello" / 2;
```

```
2
```

```
3 console.log(result);
```

Output:

```
1 NaN
```

To check NaN:

```
1 Number.isNaN(result);
```

Q44. What is the difference between function declaration and function expression?

Answer:

Function declaration:

```
1 function greet() {  
2   console.log("Hello");  
3 }
```

Function expression:

```
1 const greet = function() {  
2   console.log("Hello");  
3 };
```

Main difference:

Function declarations are hoisted, so they can be called before they are defined.

```
1 sayHello();  
2  
3 function sayHello() {  
4   console.log("Hello");  
5 }
```

Function expressions are not used before initialization.

Q45. What is ES6?

Answer:

ES6, also called ECMAScript 2015, is a major update to JavaScript.

Important ES6 features:

- *let* and *const*
- Arrow functions
- Template literals

- Destructuring
- Spread and rest operators
- Classes
- Promises
- Modules

Interview line:

ES6 introduced modern JavaScript features that make code cleaner and easier to write.